



Setup & Run Guide

Everything needed to take this repository from a fresh clone to a running application — the local development path, and the single-container Docker path.

PATH A

Terminal

Hot reload for day-to-day development. API on 8000, UI on 3000.

PATH B

Docker

One production-like container on 8382, MongoDB bundled in.

01 Prerequisites

LAMA is a FastAPI backend, a React 19 frontend, and MongoDB as its system of record. Only MongoDB is strictly required to boot — everything else unlocks a specific capability.

Software	Needed for	Install
Python 3.11+ 3.14 ok	Backend runtime	<code>brew install python@3.14</code>
MongoDB 7 / 8	Required — system of record	<code>brew tap mongodb/brew && brew install mongodb-community</code>
Node 18+	Frontend build & dev server	<code>brew install node</code>
yarn 1.22	Frontend packages — never npm	<code>corepack enable</code>
LLM provider	All generation features	Ollama (below) or an OpenRouter key
Maven · Gradle Go · .NET	CodeGen compile & test agent	<code>brew install maven gradle go dotnet</code>
Poetry · pnpm	Additional build tools	<code>brew install poetry</code> · <code>corepack enable pnpm</code>
Docker Desktop	Path B only	docker.com/products/docker-desktop

A missing build tool is a hard failure, not a skip. The Tester/Compiler agent returns `status: "failed"` when a binary is absent — deliberately, so a build can never report `compilation_ready: true` without having really compiled. Install the toolchains for the languages you actually generate.

02 Choosing an LLM provider

Without one the application still boots and serves every page, but each generation request returns *“no generation provider is healthy”*. Pick either option below — you do not need both.

Ollama — local, free, offline

Also supplies embeddings, so you never need to download PyTorch.

```
# install & start
brew install ollama
ollama serve          # :11434

# models
ollama pull qwen3:4b      # 262K
ollama pull qwen2.5-coder:7b # code
ollama pull qwen3:0.6b    # fast
ollama pull nomic-embed-text # vectors
```

OpenRouter — cloud

One key unlocks many hosted models. Add it to `backend/.env`.

```
OPENROUTER_API_KEY=sk-or-...
```

Register the provider

LAMA reads providers from the **database**, not from environment variables. Add it once in the UI under **Console → Models**, then set the `low` / `medium` / `high` routing tiers.

Verify the provider is live

The API reports exactly which provider it chose, and why each alternative was rejected — the fastest way to diagnose a silent generation failure.

```
$ curl -s http://127.0.0.1:8000/api/health/providers

{
  "generation": {
    "chosen": "ollama",
    "detail": "ollama ready (reachable at http://localhost:11434)",
    "checks": [
      { "provider": "factory", "healthy": false,
        "reason": "droid binary not found in PATH" },
      { "provider": "ollama", "healthy": true,
        "reason": "reachable at http://localhost:11434" }
    ]
  }
}
```

Thinking models need room. The Qwen3 family spends 300–1600 tokens reasoning before emitting any content, so a small token budget returns an empty response. Allow **2000+** tokens, or route short latency-sensitive calls to `qwen2.5-coder:7b`, which does not think.

03 Run from the terminal

1 Clone and create the virtual environment

The virtual environment lives at the repository root and is named `.venv`.

```
$ git clone <your-repo-url> lama
$ cd lama

$ python3 -m venv .venv
$ ./venv/bin/python -m pip install --upgrade pip
$ ./venv/bin/python -m pip install -r backend/requirements-dev-macos.txt
```

2 Install frontend packages

```
$ corepack enable          # provides yarn 1.22.22
$ cd frontend && yarn install && cd ..
```

Shortcut. `./scripts/setup.sh` performs steps 1 and 2 and writes a `backend/.env` with a freshly generated JWT secret.

3 Configure `backend/.env`

`server.py` loads `backend/.env` — not the repo-root `.env`, which belongs to docker-compose. It is gitignored.

```
MONGO_URL=mongodb://127.0.0.1:27017    # required
DB_NAME=lama                          # required
LAMA_JWT_SECRET=<48+ random chars>

LAMA_OLLAMA_BASE_URL=http://localhost:11434/v1
LAMA_DEFAULT_MODEL=                   # keep EMPTY
LAMA_CONFIDENCE_ENGINE=fabric         # pin explicitly

QDRANT_PATH=./qdrant_local            # offline vectors
LAMA_EMBED_BACKEND=ollama
LAMA_OLLAMA_EMBED_MODEL=nomic-embed-text
```

Two settings worth understanding. Any value in `LAMA_DEFAULT_MODEL` overrides per-agent tier routing for *every* call, so Console routing silently stops working — leave it empty. And in `auto` mode the confidence engine flips to LangGraph whenever the `droid` CLI is unreachable *and* `langgraph` happens to be importable, so merely installing a package changes how artifacts are scored. Pin it.

4

Start MongoDB

```
$ brew services start mongodb-community
$ nc -z 127.0.0.1 27017 && echo "Mongo up"
```

5

Run the two processes

TERMINAL 1 – API

```
$ cd backend
$ ../.venv/bin/uvicorn server:app --reload --port 8000
```

TERMINAL 2 – UI

```
$ cd frontend
$ REACT_APP_BACKEND_URL=http://127.0.0.1:8000 yarn start
```

Or use `./scripts/run-backend.sh` and `./scripts/run-frontend.sh`.

URL	What you get
<code>http://localhost:3000</code>	The application
<code>http://127.0.0.1:8000/api/health</code>	<code>{"ok":true}</code>
<code>http://127.0.0.1:8000/api/health/providers</code>	Which LLM provider is live
<code>http://127.0.0.1:8000/docs</code>	Interactive API documentation

Keep hostnames consistent. Browsers treat `localhost` and `127.0.0.1` as different origins. Mixing them between the UI and `REACT_APP_BACKEND_URL` triggers a CORS preflight failure that looks like a server bug.

04 Run with Docker

nginx, MongoDB and the API run together in one container on **:8382**. MongoDB is bundled — no local installation needed.

Docker overwrites `backend/.env`. `docker/entrypoint.sh` writes a runtime `.env` to `/app/backend/.env`, and compose bind-mounts `./backend` there — so the write lands on your **host** file and your local settings (JWT secret, Qdrant, Ollama) are replaced with container defaults. Back it up first: `cp backend/.env backend/.env.mine`.

Apple Silicon. The published image is `linux/amd64`, so it runs under emulation on M-series Macs. It works — verified — but starts and runs more slowly than a native image would.

1 Build the UI bundle first

Compose bind-mounts `./frontend/build` into nginx read-only, so it must exist *before* the container starts.

```
$ cd frontend && yarn install && yarn build && cd ..
```

2 Choose the image

The repo-root `.env` pins `LAMA_IMAGE=lama:local` with `LAMA_PULL_POLICY=never`, so compose will not pull. Pick one:

```
# Option 1 — build locally (adds git, dulwich, JDK + Maven)
$ docker build -f Dockerfile.local -t lama:local .

# Option 2 — use the public image; comment out BOTH lines in ./env
#   LAMA_IMAGE=lama:local
#   LAMA_PULL_POLICY=never
$ docker pull mishramesh/lama:latest
```

3 Point the container at an LLM provider

The root `.env` defaults to `LAMA_FACTORY_MODE=cli`, which needs the `droid` binary *inside* the container. Without it every LLM call fails.

```
# repo-root .env
LAMA_FACTORY_MODE=           # blank — disable droid mode
LAMA_DEFAULT_MODEL=         # blank — use Console routing
LAMA_DISABLE_OPENROUTER_FALLBACK=0

# reach an Ollama running on the HOST
LAMA_OLLAMA_BASE_URL=http://host.docker.internal:11434/v1
```

4

Start it

```
$ docker compose up -d
$ docker compose logs -f lama
$ curl http://127.0.0.1:8382/health      # {"ok":true}
```

Then open `http://127.0.0.1:8382`.

Everyday commands

Command	Effect
<code>docker compose restart lama</code>	Apply <code>backend/*.py</code> edits (bind-mounted)
<code>(cd frontend && yarn build) && docker compose restart lama</code>	Apply UI edits
<code>docker compose down</code>	Stop, keep data volumes
<code>docker compose down -v</code>	Stop and delete MongoDB data
<code>docker compose ps</code>	Status and health
<code>docker compose exec lama bash</code>	Shell inside the container

05 Scripts, tests & troubleshooting

Scripts in this repository

Script	What it does
<code>./scripts/setup.sh</code>	Creates <code>.venv</code> , installs backend + frontend dependencies, writes <code>backend/.env</code>
<code>./scripts/doctor.sh</code>	Checks MongoDB, Ollama, build toolchains, Qdrant, Docker — read-only
<code>./scripts/run-backend.sh</code>	Starts the API with hot reload
<code>./scripts/run-frontend.sh</code>	Starts the UI on :3000

Tests

```
$ ./venv/bin/python -m pytest backend/tests/ -q
$ ./venv/bin/python -m pytest backend/tests/test_lama_v4.py -k test_chat -x
$ ./venv/bin/python -m pyflakes backend/routes/codegen.py

# suites that call a live server need one running:
$ REACT_APP_BACKEND_URL=http://127.0.0.1:8000 \
  ./venv/bin/python -m pytest backend/tests/ -q
```

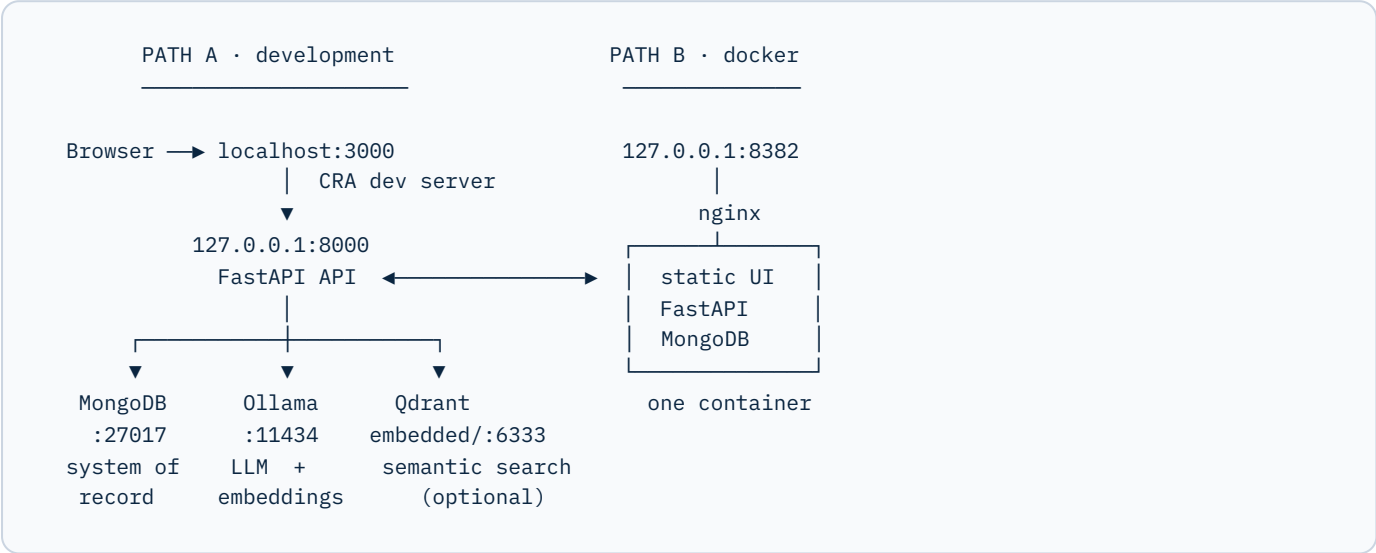
Expected failures. Suites written before multi-tenant auth call the API without a bearer token and fail with `401` . That is the API behaving correctly, not a regression.

Troubleshooting

Symptom	Cause & fix
<code>KeyError: 'MONGO_URL'</code>	<code>backend/.env</code> missing — it must sit beside <code>server.py</code>
Starts, then hangs	MongoDB unreachable; the startup seed is waiting. Start Mongo.
<i>no generation provider is healthy</i>	No LLM configured. Check <code>/api/health/providers</code> .
LLM returns empty content	Thinking model exhausted its budget. Raise <code>max_tokens</code> to 2000+.
<code>401 Unauthorized</code>	Expected — log in. LAMA is multi-tenant with JWT auth.
<code>toolchain missing on PATH</code>	Install it; absence fails the job by design.
CORS / missing <code>Allow-Origin</code>	<code>localhost</code> vs <code>127.0.0.1</code> mismatch

<code>library load disallowed by system policy</code>	macOS quarantined a downloaded copy: <code>xattr -dr com.apple.quarantine .</code> then rebuild <code>.venv</code>
<code>pull access denied for lama:local</code>	Image not built — see Path B, step 2
Semantic search returns nothing	Qdrant off. Set <code>QDRANT_PATH</code> or <code>QDRANT_URL</code> ; it degrades silently by design.

How the pieces fit



The five-stage pipeline — **Discovery** → **DataModel** → **Architecture** → **CodeGen** → **Living** — is strictly sequential: each stage refuses to run until the one before it has been frozen.